DECA Lab Logbook

Part 2 | Section 1: Memory

Section 1: Memory

Aim

To design and simulate a synchronous ROM in Issie that stores a short ASCII text message, and to build a counter-driven message writer that reads out characters sequentially and automatically resets.

Before the Lab — ASCII Encoding

Text in digital systems is represented by mapping each character to a binary code. The ASCII standard defines 7-bit codes for the 128 characters of the English alphabet, digits and common symbols. The first 128 characters of the Unicode standard are identical to ASCII.

The message stored in ROM is "DECA GOD" (8 characters including the space). Each character is encoded as an 8-bit hexadecimal value:

Character	D	E	C	A	(space)	G	O	D
Hex code	0x44	0x45	0x43	0x41	0x20	0x47	0x4F	0x44

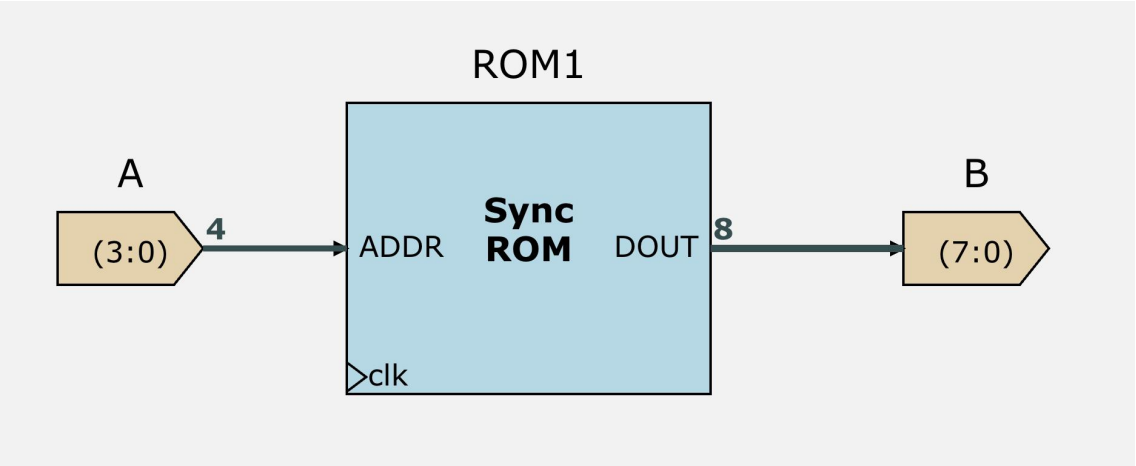
Initialising ROM

Aim

Design and test a synchronous ROM that stores an 8-character text message. The ROM outputs the character stored at the address provided, updated on every rising clock edge.

Components

- 4-bit address input A(3:0)
- Synchronous ROM configured in Issie: 4-bit address, 8-bit word width
- 8-bit data output B(7:0)



How It Works

The 8 characters are entered into the ROM memory editor in ASCII hexadecimal format (addresses 0x0 through 0x7). Address 0x8 onward contains 0x00 (null).

Because the ROM is synchronous, the output only updates on the rising edge of the clock — it

behaves like a D-type flip-flop for the data output. When an address value 0–7 is presented at input A and the clock is ticked, the corresponding character code appears on output B.

Address	Content (hex)	Character
x0	x44	D
x1	x45	E
x2	x43	C
x3	x41	A
x4	x20	(space)
x5	x47	G
x6	x4F	O
x7	x44	D
x8	x00	(null terminator)

Simulating the ROM

Input and output ports are added to the ADDR and DOUT pins respectively. Using step simulation, each of the 16 possible address values (0x0–0xF) is applied in turn. After each clock tick the output updates to the value stored at the given address.

Addresses 0–7 correctly output the ASCII codes for 'D', 'E', 'C', 'A', ' ', 'G', 'O', 'D'. Addresses 8–15 output 0x00 (uninitialised / null).

Section 1 Challenge: Message Writer

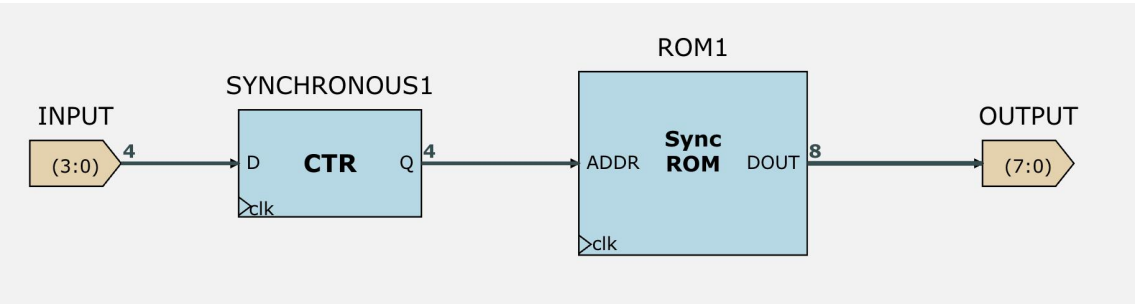
Message Writer (Auto-cycling)

Aim

Design a system that automatically cycles through all characters stored in ROM and repeats, without any user input. The auto-reset counter drives the ROM address, incrementing every clock tick and resetting to 0 after the last character.

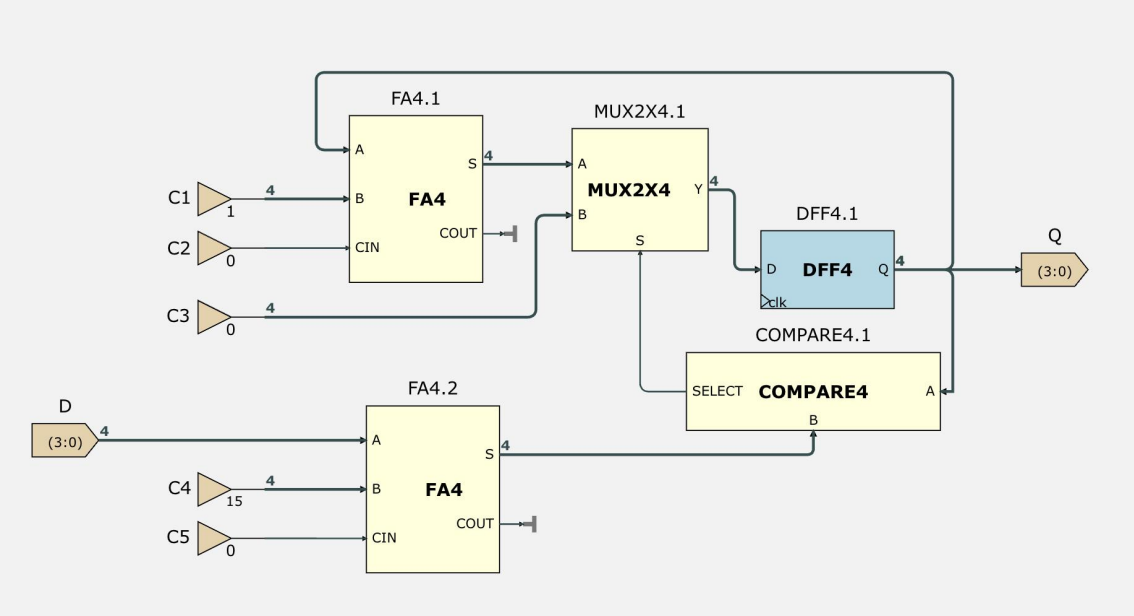
Components

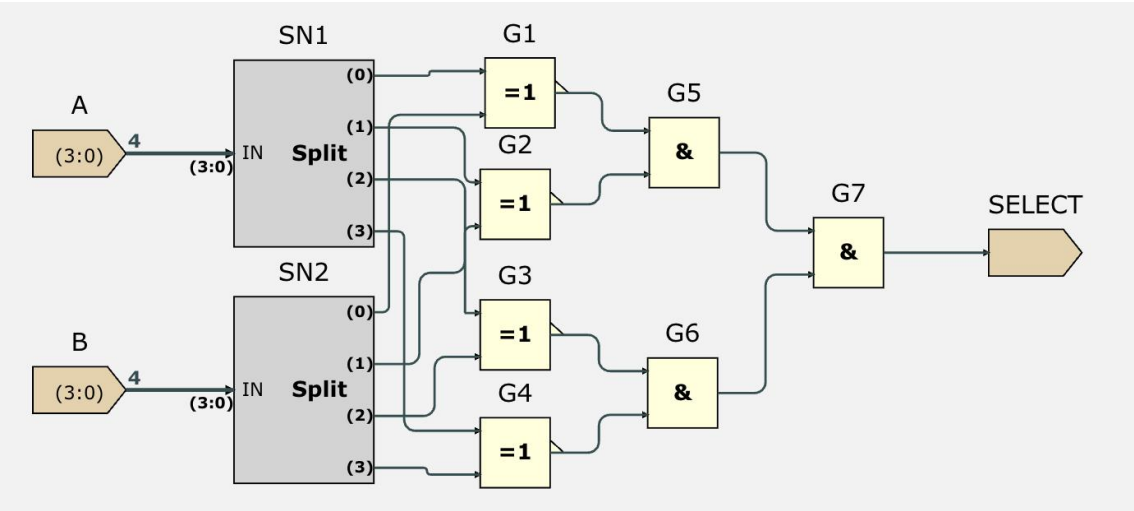
- Synchronous ROM (storing 8 characters: DECA GOD)
- Auto counter CTR (drives the ADDR input of ROM)
- 4-bit input D (sets message length limit), 8-bit output



Auto Counter CTR — Internal Components

- FA4: 4-bit adder, adds 1 to the current count each tick
- FA4 (subtractor): computes D − 1 to find the last valid address
- COMPARE4: 4-bit comparator — outputs 1 when Q = D − 1
- MUX2X4: selects either Q + 1 or constant 0 based on comparator output
- DFF4: 4-bit register storing the current counter value





How It Works

The value stored in DFF4 starts at 0. After each clock tick, Q is output and simultaneously fed through FA4 to produce Q + 1. D is fed through the subtractor to give D − 1 (since the ROM address starts at 0, the length input needs to be decremented to find the last address).

While Q < D − 1: COMPARE4 outputs 0, so MUX selects the adder output (Q + 1), which is stored in DFF4 on the next clock tick.

When Q = D − 1: COMPARE4 outputs 1, so MUX selects constant 0. On the next clock tick, DFF4 latches 0 and the cycle restarts.

Setting D to the number of characters in the message causes the ROM address to cycle through 0, 1, 2, … D−1, 0, 1, … automatically. Since the ROM is synchronous, there is a 1-tick delay between the counter output and the ROM data output, visible in wave simulation.

Message Writer with Trigger Input

Aim

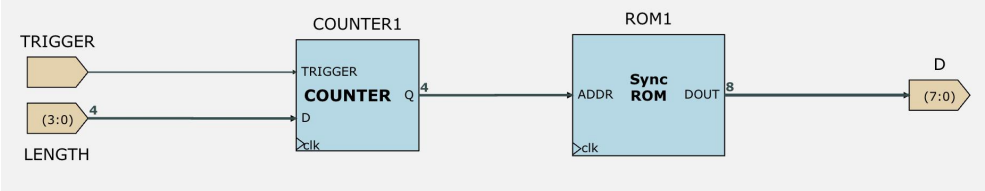
Extend the message writer so that the counter only advances when a 1-bit TRIGGER input is high. When TRIGGER is low and Q = 0, the counter freezes at address 0 and only the first character is output.

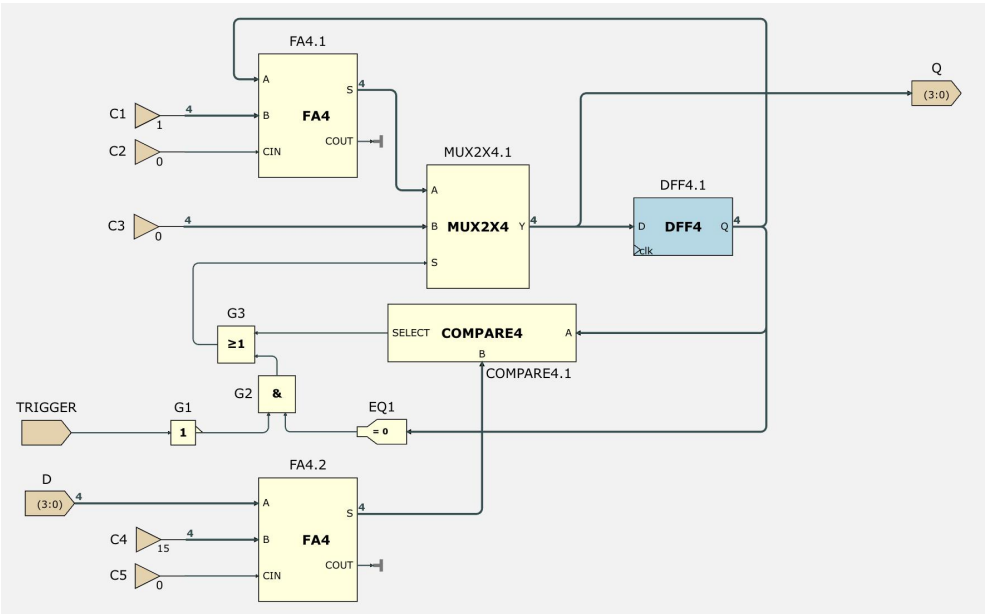
Counter Behaviour Table

Counter value Q	Condition	Next value
Q = 0	Trigger low	0 (freeze)
Q = 0	Trigger high	Q + 1
0 < Q < N − 1	Always	Q + 1
Q = N − 1	Always	0 (auto-reset)

Components of the Counter (Trigger Version)

- FA4 (adder): computes Q + 1
- MUX2X4: selects Q + 1 or constant 0 as the next DFF4 input
- DFF4: 4-bit register; note — Q is taken from the MUX output, not DFF4 output, to avoid a 1-tick delay when trigger transitions
- COMPARE4 (end comparator): outputs 1 when Q = D − 1 (last address)
- COMPARE4 (zero comparator): outputs 1 when Q = 0
- NOT gate: inverts TRIGGER
- AND gate: combines NOT(TRIGGER) and zero-comparator output to produce freeze signal
- OR gate: combines freeze signal and end-comparator output to drive MUX select S





How It Works

The MUX select S is driven high (select constant 0) by two conditions, combined with an OR gate:

1. End of message: COMPARE4 (end) outputs 1 when Q = D − 1 — always resets to 0.
2. Freeze at start: AND(NOT(TRIGGER), COMPARE4(zero)) outputs 1 when trigger is low AND Q = 0 — holds at 0.

In all other cases S = 0 and the adder output Q + 1 is latched into DFF4 on the next clock tick.

Q is taken from the MUX output (not DFF4 directly) to prevent a 1-tick propagation delay when the trigger transitions from low to high. If DFF4 output were used, changing the trigger would require an extra clock tick before the counter begins advancing, because the trigger change cannot immediately alter the value already stored in DFF4.

When TRIGGER is high and Q = 0: NOT(TRIGGER) = 0, so the AND gate outputs 0, S = 0, and the counter starts advancing normally.

When TRIGGER is low and Q = 0: NOT(TRIGGER) = 1 and COMPARE4(zero) = 1, so AND = 1, OR = 1, S = 1, and MUX selects 0 — the counter stays frozen.